

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: B.Charishma

Roll No.: a24126552067

Date: 30-08-2026

1. TITLE

Object-Oriented Programming and Data Encapsulation using JavaScript ES6 Classes

2. AIM

To implement object-oriented programming principles in JavaScript by creating ES6 classes, instantiating objects, and dynamically injecting structured data into the Document Object Model (DOM).

3. OBJECTIVE

The objectives of this experiment are:

- To define JavaScript ES6 classes with constructors.
- To instantiate objects encapsulating student profile attributes.
- To extract user input from HTML form elements using JS.
- To validate form inputs dynamically preventing empty submissions.
- To construct HTML template strings and inject them into the DOM using innerHTML.

4. THEORY

JavaScript ES6 introduced the `class` keyword, providing syntactic sugar over the existing prototype-based inheritance model. Classes provide a much clearer and simpler syntax to create objects and deal with inheritance.

A constructor is a special method for creating and initializing an object created with a class. Data encapsulation is achieved by grouping attributes (like name, roll number, CGPA) into a single object instance.

The Document Object Model (DOM) allows JavaScript to read values from input fields, execute conditional validation logic, and dynamically update the page content without requiring a server request.

Application Flow:

User Input -> Button Click Event -> Form Validation -> Class Instantiation -> DOM Injection

5. ALGORITHM / PROCEDURE

1. Create a simple HTML UI containing input fields for Name, Roll No, Department, and CGPA.
2. Add a "Show Profile" button that triggers a JavaScript function on click.
3. Define an ES6 class named Student with a constructor initializing 4 properties.
4. Inside the click handler, retrieve values from the input fields using `document.getElementById().value`.
5. Implement an IF condition to check for empty strings and trigger an alert if validation fails.
6. Instantiate a new Student object passing the retrieved values as arguments.
7. Construct a template literal string mapping the object properties into HTML markup.
8. Inject the resulting string into a target `<div>` using the `innerHTML` property.
9. Test edge cases such as empty submissions and extreme values.

6. PROGRAM / SOURCE CODE

Source File: task3.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Basic Student Profile</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
      background: #f5f5f5;
    }
    .container {
      max-width: 400px;
      margin: auto;
      background: white;
      padding: 25px;
      border-radius: 10px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    input {
      width: 100%;
      padding: 8px;
      margin: 5px 0 15px 0;
      border: 1px solid #ddd;
      border-radius: 5px;
      box-sizing: border-box;
    }
    button {
      width: 100%;
      padding: 10px;
      background: #007bff;
      color: white;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      font-size: 16px;
    }
    button:hover {
      background: #0056b3;
    }
    .profile {
      margin-top: 20px;
      padding: 15px;
      background: #f9f9f9;
      border-radius: 5px;
      border-left: 4px solid #007bff;
    }
    .profile p {
      margin: 8px 0;
    }
  </style>
</head>
<body>
  <div class="container">
    <input type="text" value="Name" />
    <input type="text" value="Email" />
    <input type="text" value="Phone" />
    <button type="button" value="Save Profile" />
  </div>
  <div class="profile">
    <p>Name: John Doe</p>
    <p>Email: john.doe@example.com</p>
    <p>Phone: +1 234 567 890</p>
  </div>
</body>
</html>
```

```

        .label {
            font-weight: bold;
            display: inline-block;
            width: 100px; /* Keeps the colons aligned */
        }
    </style>
</head>
<body>

<div class="container">
    <h2>Student Profile</h2>

    <label>Name</label>
    <input type="text" id="name" value="B.Charishma">

    <label>Roll Number</label>
    <input type="text" id="roll" value="A24126552067">

    <label>Department</label>
    <input type="text" id="dept" value="CSE (AI & ML)">

    <label>CGPA</label>
    <input type="text" id="cgpa" value="8.42">

    <button onclick="showProfile()">Show Profile</button>

    <div id="profileDisplay"></div>
</div>

<script>
    // 1. Create the Student class with properties
    class Student {
        constructor(name, rollNumber, department, cgpa) {
            this.name = name;
            this.rollNumber = rollNumber;
            this.department = department;
            this.cgpa = cgpa;
        }
    }

    // 2. The function that runs when the button is clicked
    const showProfile = () => {

        // Get user input values from the text boxes
        let name = document.getElementById('name').value;
        let roll = document.getElementById('roll').value;
        let dept = document.getElementById('dept').value;
        let cgpa = document.getElementById('cgpa').value;

        // Validation - stop if fields are empty
        if (name === '' || roll === '' || dept === '' || cgpa === '') {
            alert('Please fill in all fields.');
```

```

        return;
    }

    // 3. Create a new Student object
    const studentData = new Student(name, roll, dept, cgpa);

    // 4. Display profile dynamically using simple innerHTML
    const displayDiv = document.querySelector('#profileDisplay');
    displayDiv.innerHTML = `
        <div class="profile">
            <p><span class="label">Name</span> : ${student.name}</p>
            <p><span class="label">Roll No</span> : ${student.rollNumber}</p>
            <p><span class="label">Department</span> : ${student.department}</p>
            <p><span class="label">CGPA</span> : ${student.cgpa}</p>
        </div>
    `;
}
</script>

</body>
</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>class Student { constructor(...) }</code>	Defines an ES6 class blueprint with an initialization function.
<code>let student = new Student(...)</code>	Instantiates a new object from the Student class with specific data.
<code>document.getElementById('id').value</code>	Retrieves the current string value from an HTML input element.
<code>if (name === '') { alert(...); return; }</code>	Validates input and halts function execution if criteria are not met.
<code>display.innerHTML = `... \${var} ...`</code>	Updates the DOM content of an element dynamically using template literals.

8. TEST CASES AND EXECUTION DATA

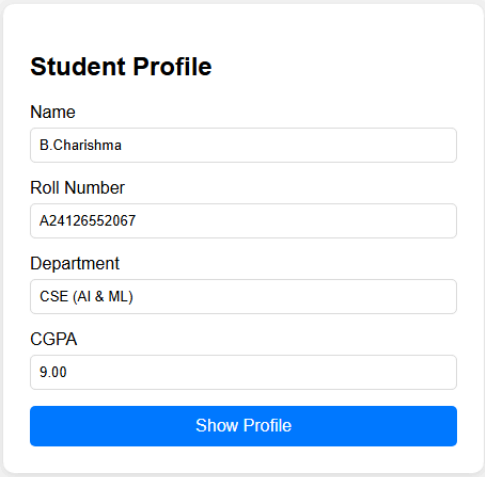
The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
-----------	----------------	-----------------	---------------	--------

TC-01	Valid Object Instantiation	Provide all student details and click Show Profile	Profile card rendered dynamically with correct data	Pass
TC-02	Data Update & Re-render	Change CGPA to 9.00 and submit again	DOM successfully replaces old card with updated 9.00 card	Pass
TC-03	Empty Form Submission	Clear all inputs and click Show Profile	Validation halts execution and fires browser alert	Fail

10. TEST CASES SCREENSHOTS

Test Case Execution 1



The screenshot displays a 'Student Profile' form within a light gray container. The form is a white card with rounded corners and a subtle shadow. It contains four text input fields, each with a label above it: 'Name' (containing 'B.Charishma'), 'Roll Number' (containing 'A24126552067'), 'Department' (containing 'CSE (AI & ML)'), and 'CGPA' (containing '9.00'). Below these fields is a solid blue button with the text 'Show Profile' in white.

Field	Value
Name	B.Charishma
Roll Number	A24126552067
Department	CSE (AI & ML)
CGPA	9.00

11. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.